

레이아웃과 프래그먼트

학습 체크리스트 (1/2)

☐ 공통 화면 조각을 프래그먼트로 분리해야 하는 이유와 폴더 구성 규약 이해

☐ `th:fragment` 정의와 `~{경로 :: 이름}` 참조 문법 습득

☐ `th:replace` · `th:insert` · `th:remove` 의 동작 차이 습득

☐ 파라미터를 받는 프래그먼트와 `th:block` 활용법 습득

학습 체크리스트 (2/2)

☐ 순수 Fragment Expression 방식으로 레이아웃을 구성하는 방법 이해

☐ thymeleaf-layout-dialect의 `layout:decorate` · `layout:fragment` 구조 이해

☐ 공통 `head` 프래그먼트와 동적 title·메타 태그 구성 습득

☐ 정적 리소스 경로와 `@{...}` URL 표현식의 필요성 이해

프래그먼트로 화면 조각 재사용

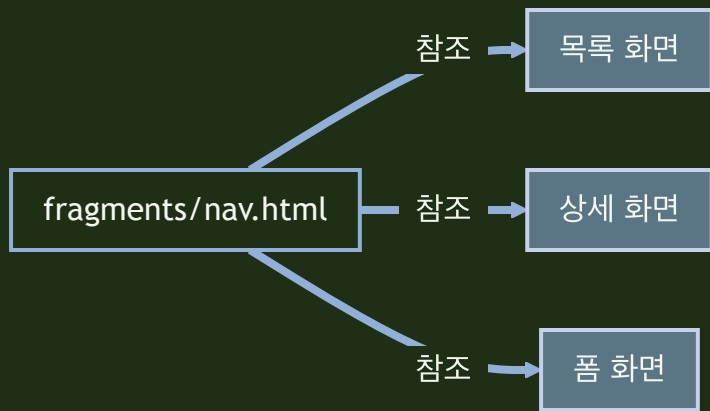
화면마다 반복되는 공통 조각

- 목록·상세·폼 화면은 각각 `<head>`, 네비게이션, 푸터를 담고 있음
- 공통 부분을 화면마다 복사해 두면 메뉴 하나만 바뀌도 모든 템플릿을 일일이 고쳐야 함
- 공통 조각을 한 곳에 정의하고 여러 템플릿에서 가져다 쓰면 수정 지점이 하나로 줄어듦

프래그먼트 (Fragment)

여러 템플릿에서 이름으로 불러와 재사용할 수 있도록 이름 붙여 둔 HTML 조각

복사 대신 참조로 조각을 공유하는 구조



프래그먼트 사용에 필요한 의존성

- 프래그먼트 정의·참조는 Thymeleaf 표준 기능이며 별도 레이아웃 라이브러리가 필요하지 않음
- Spring Boot 애플리케이션에서는 `spring-boot-starter-thymeleaf`만 추가하면 바로 사용 가능
- 여러 화면에 걸친 레이아웃 상속용 서드파티인 `thymeleaf-layout-dialect`는 뒤에서 다룸

역할별 템플릿 폴더 구성

```
1 templates/  
2   layout/      (전체 골격)  
3   fragments/   (head, nav, footer 같은 공통 조각)  
4   books/       (화면별 템플릿)
```

- 폴더 이름은 프레임워크의 필수 규칙이 아니라 프로젝트에서 정하는 관례
- 프래그먼트와 완성 화면을 역할별 폴더로 구분하면 경로 충돌을 줄이고 유지보수하기 쉬움

templates 경로가 렌더링되는 방식

- `templates/`는 정적 리소스처럼 브라우저가 바로 접근하는 공개 경로가 아님
- 컨트롤러가 반환한 뷰 이름을 서버가 찾아 렌더링하는 내부 경로
- 프래그먼트 전용 폴더를 분리하면 완성된 화면과 부품을 구분하기 쉬움

th:fragment로 조각 정의하기

```
1 <!-- fragments/nav.html -->
2 <nav th:fragment="mainNav">
3   <a th:href="@{/books}">도서 목록</a>
4 </nav>
```

프래그먼트 참조 문법 정리

형태	의미
<code>~{경로 :: 이름}</code>	다른 템플릿에 정의된 프래그먼트 참조
<code>~{:: 이름}</code>	같은 템플릿 안의 프래그먼트 참조
<code>~{경로 :: #선택자}</code>	이름 대신 CSS 선택자로 영역 지정

- Spring Boot가 구성한 Thymeleaf 템플릿 리졸버는 기본 suffix인 `.html`을 템플릿 이름 뒤에 붙여 탐색
- 기본 설정에서는 `~{fragments/nav :: mainNav}`처럼 확장자를 생략하며, suffix 설정을 바꾸면 작성 규칙도 달라짐

th:insert·th:replace·th:remove 비교

속성	동작	비고
th:replace	호스트 태그 자체를 프래그먼트로 교체	불필요한 래퍼를 남기지 않을 때 적합
th:insert	호스트 태그는 남기고 안쪽에 프래그먼트 삽입	감싸는 태그가 필요할 때
th:remove	렌더링 시 태그를 제거	all · body · tag · all-but-first 등

- th:replace 와 th:insert 를 혼동하여 태그가 불필요하게 2중 중첩되거나 의도한 마크업 구조가 깨져 화면 레이아웃이 붕괴하는 조작 실수가 많으므로 주의 요망

replace와 insert의 결과 차이

```
1 <div th:replace="~{fragments/nav :: mainNav}"></div>
2 <div th:insert="~{fragments/nav :: mainNav}">여기 안쪽에 삽입됩니다</div>
```

th:remove 값별 삭제 범위

값	삭제 범위
all	태그와 자식 모두 삭제
body	자식만 삭제, 태그는 유지
tag	태그만 삭제, 자식은 상위로
all-but-first	첫 자식만 남기고 나머지 삭제
none	삭제하지 않음

정적으로 열어볼 때만 필요한 더미 제거

- 디자이너가 작업한 시안용 마크업 등 정적 확인 용도로만 필요한 더미 요소에 `th:remove="all"` 선언을 생략하면 서버 동작 시 더미가 그대로 노출되는 실수가 생김
- `th:remove="all"` 을 적절히 설정해야 서버가 렌더링할 때 더미를 제거하고 순수 데이터만 표출하며, 브라우저로 직접 HTML을 열었을 때만 더미가 정상 노출됨

비권장 속성 (Not Recommended)

`th:include` 는 Thymeleaf 3.0부터 권장되지 않으며 `th:insert` 나 `th:replace` 로 대체함

값을 외부에서 주입받는 프래그먼트 정의

```
1  <!-- fragments/card.html -->
2  <div th:fragment="card(title, url)">
3    <a th:href="${url}" th:text="${title}">제목</a>
4  </div>
```


파라미터를 채워 프래그먼트 호출하기

```
1 <div th:replace="~{fragments/card :: card('클린 코드', '/books/1')}"></div>
```

이름 기반 인자로 프래그먼트 호출하기

- 위치 순서 대신 파라미터 이름을 지정해 값을 전달 가능
- 인자가 많아질수록 어떤 값이 어떤 파라미터인지 명확해짐

```
1 <div th:replace="~{fragments/card :: card(title=${book.title}, url=@{/books/{id}}(id=${book.id}))"
```

th:block으로 태그 없이 묶기

- 실제 HTML 태그로 렌더링되지 않는 논리 블록
- 반복·조건을 감쌀 때 불필요한 `<div>` 를 만들지 않고 여러 요소를 그룹으로 다룸

th:block으로 반복 렌더링하기

```
1 <th:block th:each="book : ${books}">
2   <dt th:text="${book.title}">제목</dt>
3   <dd th:text="${book.author}">저자</dd>
4 </th:block>
```

브라우저로 바로 열어도 깨지지 않는 이유

내추럴 템플릿 (Natural Templates)

서버에서 처리하지 않아도 HTML 파일을 정적 프로토타입으로 열어 화면의 기본 구조와 예시 내용을 확인할 수 있는 성질

- 브라우저는 `th:*` 속성을 실행하지 않고 기본 HTML 마크업과 예시 내용을 표시하므로 로컬에서 화면 구조를 확인할 수 있음
- 프래그먼트 조각 결합과 실제 서버 데이터 바인딩은 서버 사이드 엔진을 거쳐 렌더링될 때만 작동

레이아웃 구성 전략

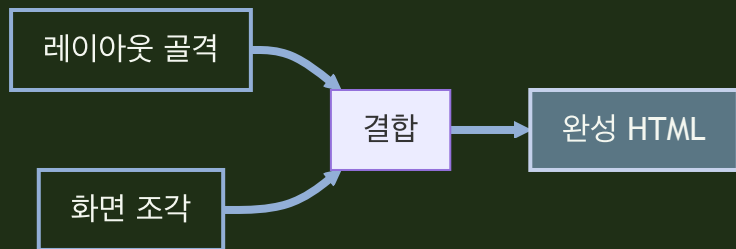
화면마다 반복되는 공통 마크업 문제

- 여러 화면이 공통 헤더·네비게이션·푸터를 공유하는 경우가 대부분
- 매 템플릿에 같은 마크업을 그대로 복사하면 변경 시 모든 파일을 함께 고쳐야 함
- Thymeleaf 진영에는 이 문제를 푸는 두 가지 접근법이 존재함
- 의존성 유무와 호출부 형태에서 두 방식이 갈림

레이아웃 구현 두 방식 비교

방식	추가 의존성	특징
순수 Thymeleaf (Fragment Expression)	없음	파라미터를 받는 프래그먼트로 레이아웃을 정의하고, 화면이 자기 조각을 인자로 넘김
thymeleaf-layout-dialect	있음 (<code>nz.net.ultraq.thymeleaf</code>)	<code>decorate</code> / <code>fragment</code> 로 레이아웃 상속을 표현하고 <code><head></code> 를 자동 병합

레이아웃과 화면이 결합되는 흐름



Fragment Expression으로 레이아웃 정의하기

- 레이아웃 템플릿을 `title`, `content` 처럼 파라미터를 받는 프래그먼트로 정의
- 각 화면은 렌더링될 때 자기 자신의 일부 요소를 프래그먼트 표현식으로 만들
- 만들어진 조각을 레이아웃 프래그먼트의 인자로 그대로 넘김
- 별도 의존성 없이 Thymeleaf 표준 문법만으로 동작함

파라미터를 받는 레이아웃 템플릿

```
1  <!-- layout/base.html -->
2  <html th:fragment="layout(title, content)">
3  <head><title th:replace="${title}">기본 제목</title></head>
4  <body>
5    <header>공통 헤더</header>
6    <div th:replace="${content}">본문 자리</div>
7  </body>
8  </html>
```

- 레이아웃 템플릿 내에서 인자로 넘겨받은 프래그먼트 조각 객체를 변수식 `${content}` 이 아닌 문자열 리터럴 ``${content}`` 형식으로 참조하는 실수가 자주 나타남
- 잘못된 리터럴 표기 시 조각 자체가 아닌 고정 텍스트로 오작동하여 화면이 깨지므로 수식 작성을 엄격히 준수해야 함

JSP와 Thymeleaf의 조각 재사용 방식 비교 (1/2)

축	JSP	Thymeleaf
조각 포함	<code><jsp:include></code> / <code><%@ include %></code>	<code>th:replace</code>
레이아웃 상속	Apache Tiles·SiteMesh 같은 별도 프레임워크 필요	Thymeleaf 자체 기능 또는 layout-dialect

JSP와 Thymeleaf의 조각 재사용 방식 비교 (2/2)

축	JSP	Thymeleaf
정적 열람	서버 실행 없이는 브라우저에서 바로 열 수 없음	내추럴 템플릿(Natural Templates) 성질을 통해 HTML 파일 그대로 열람 가능
실행 방식	서블릿으로 컴파일되어 실행	템플릿 엔진이 런타임에 파싱
표현식	EL·JSTL	Thymeleaf 표준 표현식

정적 포함과 동적 포함의 차이

- `<%@ include %>`는 JSP 변환 시점에 대상 파일의 내용을 소스에 합치는 정적 포함
- `<jsp:include>`는 런타임에 요청을 위임하는 동적 포함
- Thymeleaf의 `th:replace`도 렌더링 시점에 동작하지만, 별도 요청 위임 없이 같은 템플릿 처리 과정에서 조각을 치환함

Spring Boot에서 JSP를 권장하지 않는 이유

- Spring Boot는 실행 가능한 JAR로 내장 톰캣을 띄우는 방식을 기본으로 함
- 실행 가능한 JAR에서는 JSP가 지원되지 않으며, JSP를 사용하려면 일반적으로 WAR 패키징과 지원되는 서블릿 컨테이너 구성이 필요함
- Spring Boot 공식 문서도 내장 컨테이너 환경의 제약 때문에 가능하면 JSP를 피하도록 안내함

학습 요약 (1/2)

- 프래그먼트로 화면 조각 재사용:

- 공통 조각을 `th:fragment` 로 정의하고 `~{경로 :: 이름}` 으로 참조해 수정 지점을 하나로 모으는 방식 습득

- `th:replace`·`th:insert`·`th:remove`의 차이:

- 세 속성이 호출 태그와 프래그먼트 태그를 다루는 방식이 다르다는 점과 각각의 쓰임을 구분하는 방법 이해

- 레이아웃 구성 전략:

- 순수 Fragment Expression 방식과 `thymeleaf-layout-dialect`의 `layout:decorate` · `layout:fragment` 구조 이해

학습 요약 (2/2)

- 공통 head와 메타 태그:
 - 여러 화면이 공유하는 `head` 프래그먼트를 만들고 화면별 title·메타 태그를 동적으로 채우는 방법 습득
- 정적 리소스와 URL 표현식:
 - `@{...}` 표현식으로 컨텍스트 경로가 반영된 정적 리소스 경로를 구성하는 방법 이해
- JSP와의 비교:
 - 조각 포함·레이아웃 상속·정적 열람·실행 방식에서 JSP와 Thymeleaf가 갈라지는 지점 이해